Лабораторная работа №6

Декораторы, контекстные менеджеры и расширенные возможности Python

1. Цель работы

Освоить dataclass, Enum, декораторы, contextmanager, Protocol и интроспекцию объектов.

2. Постановка индивидуального задания

Вариант 8. Фитнес-центр. Разработать систему клиентов, тренеров и абонементов. Статусы абонемента: активен, истёк, заморожен, отменён. Дополнительные операции — определение посещаемости и расчёт стоимости абонемента. Меню включает интроспекцию объекта.

3. Краткие теоретические сведения

dataclass сокращает шаблонный код модели. Enum задаёт фиксированный набор статусов. Декораторы добавляют журналирование, измерение времени и ограничение вызовов. contextmanager создаёт контекст операции, а dir(), type(), isinstance(), hasattr() и getattr() исследуют объект во время выполнения.

4. Исходный код программы

Файл context.py

```
from contextlib import contextmanager
from datetime import datetime
from pathlib import Path

@contextmanager
def operation_log(path="operations.log", operation="операция"):
    started = datetime.now()
    try:
        yield
    except Exception as exc:
        with Path(path).open("a", encoding="utf-8") as file:
            file.write(f"{started.isoformat()} | {operation} | ошибка: {exc}\n")
        raise
    else:
```

```
        with Path(path).open("a", encoding="utf-8") as file:
            file.write(f"{started.isoformat()} | {operation} | успешно\n")
```

Файл decorators.py

```
from functools import wraps
from time import perf_counter

def log_call(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print(f"[LOG] вызов {function.__name__}")
        result = function(*args, **kwargs)
        print(f"[LOG] {function.__name__} завершена")
        return result
    return wrapper

def limit_calls(max_calls):
    def decorator(function):
        calls = 0
        @wraps(function)
        def wrapper(*args, **kwargs):
            nonlocal calls
            if calls >= max_calls: raise RuntimeError("Лимит вызовов исчерпан.")
            calls += 1
            return function(*args, **kwargs)
        return wrapper
    return decorator

def measure_time(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        try: return function(*args, **kwargs)
        finally: print(f"[TIME] {function.__name__}: {perf_counter()-start:.6f} сек.")
    return wrapper
```

Файл main.py

```
from service import FitnessService

def main():
    service = FitnessService()
    while True:
        print("\n=== Фитнес-центр ===")
        print("1. Добавить клиента  2. Удалить клиента  3. Найти клиента")
        print("4. Все клиенты  5. Изменить статус абонемента  6. Отметить посещение")
        print("7. Посещаемость  8. Интроспекция клиента  0. Выход")
        choice = input("Выберите пункт: ").strip()
        try:
            if choice == "1":
                client = service.add_client(input("Имя: "), int(input("Возраст: "))); print(client)
            elif choice == "2":
                cid=int(input("ID клиента: "))
                if cid not in service.clients: raise KeyError("Клиент не найден.")
                del service.clients[cid]; print("Клиент удалён.")
            elif choice == "3": print(service.clients.get(int(input("ID клиента: ")), "Клиент не найден."))
            elif choice == "4":
                for client in service.clients.values(): print(client)
            elif choice == "5": print(service.set_membership_status(int(input("ID абонемента: ")), input("Статус
```

```
(активен/истёк/заморожен/отменён): ").strip()))
            elif choice == "6": print(service.record_visit(int(input("ID клиента: "))))
            elif choice == "7": print("Всего посещений:", service.attendance())
            elif choice == "8": service.inspect_client(int(input("ID клиента: ")))
            elif choice == "0": break
            else: print("Неизвестный пункт меню.")
        except (ValueError, KeyError, RuntimeError) as exc: print("Ошибка:", exc)

if __name__ == "__main__": main()
```

Файл models.py

```
from dataclasses import dataclass
from enum import Enum

class MembershipStatus(Enum):
    ACTIVE = "активен"
    EXPIRED = "истёк"
    FROZEN = "заморожен"
    CANCELLED = "отменён"

@dataclass
class Client:
    client_id: int
    name: str
    age: int
    visits: int = 0

    def get_report_data(self):
        return {"id": self.client_id, "name": self.name, "visits": self.visits}

@dataclass
class Trainer:
    trainer_id: int
    name: str
    specialization: str

@dataclass
class Membership:
    membership_id: int
    client_id: int
    months: int
    price: float
    status: MembershipStatus = MembershipStatus.ACTIVE

    def get_report_data(self):
        return {"id": self.membership_id, "client_id": self.client_id, "status": self.status.value}
```

Файл service.py

```
from models import Client, Trainer, Membership, MembershipStatus
from decorators import log_call, limit_calls, measure_time
from context import operation_log

class FitnessService:
    def __init__(self):
        self.clients = {1: Client(1, "Анна", 21, 12), 2: Client(2, "Иван", 25, 8)}
        self.trainers = {1: Trainer(1, "Олег", "силовые тренировки")}
        self.memberships = {1: Membership(1, 1, 1, 2500), 2: Membership(2, 2, 12, 22000)}
```

```
@log_call
@measure_time
@limit_calls(20)
def add_client(self, name, age):
    if not name.strip() or age < 14: raise ValueError("Имя обязательно, возраст должен быть не меньше 14 лет.")
    new_id = max(self.clients, default=0) + 1
    self.clients[new_id] = Client(new_id, name.strip(), age)
    return self.clients[new_id]

def set_membership_status(self, membership_id, status):
    if membership_id not in self.memberships: raise KeyError("Абонемент не найден.")
    if status not in {s.value for s in MembershipStatus}: raise ValueError("Неизвестный статус.")
    membership = self.memberships[membership_id]
    membership.status = next(s for s in MembershipStatus if s.value == status)
    return membership

def record_visit(self, client_id):
    if client_id not in self.clients: raise KeyError("Клиент не найден.")
    with operation_log(operation=f"посещение клиента {client_id}"):
        self.clients[client_id].visits += 1
    return self.clients[client_id]

def attendance(self):
    return sum(c.visits for c in self.clients.values())

def inspect_client(self, client_id):
    if client_id not in self.clients: raise KeyError("Клиент не найден.")
    obj = self.clients[client_id]
    print("Тип:", type(obj).__name__)
    print("Является Client:", isinstance(obj, Client))
    print("Есть visits:", hasattr(obj, "visits"))
    print("Имя через getattr:", getattr(obj, "name", "не указано"))
    print("Атрибуты:", ", ".join(name for name in dir(obj) if not name.startswith("__")))
    return obj
```

5. Объяснение работы программы

Client, Trainer и Membership — dataclass-модели. MembershipStatus перечисляет допустимые статусы. FitnessService реализует регистрацию клиента, изменение статуса и посещаемость. Методы обёрнуты декораторами, record_visit() использует контекстный менеджер, inspect_client() демонстрирует интроспекцию.

6. Примеры работы программы в виде снимков экрана

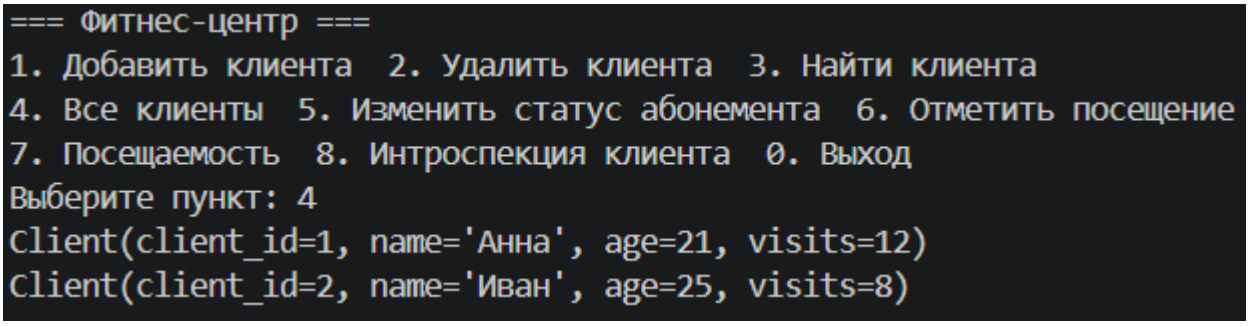


Рисунок 6.1 – Главное меню и список клиентов фитнес-центра.

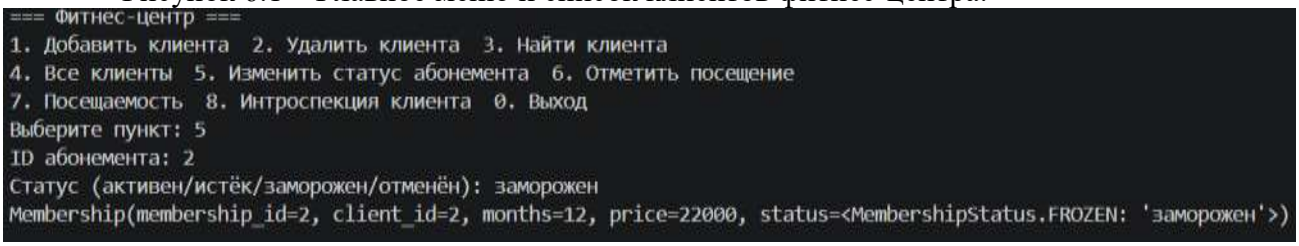


Рисунок 6.2 – Изменение статуса абонемента или регистрация клиента с журналированием.

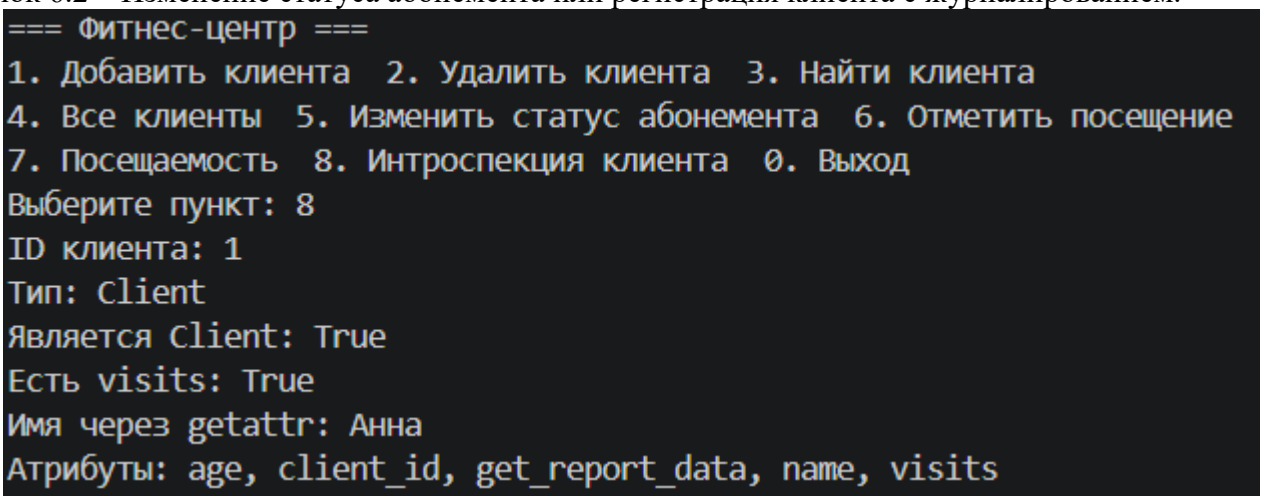


Рисунок 6.3 – Интроспекция объекта: тип, атрибуты и наличие свойств.

7. Вывод

В ходе выполнения лабораторной работы были реализованы требования индивидуального задания варианта 8. Client, Trainer и Membership — dataclass-модели. MembershipStatus перечисляет допустимые статусы. FitnessService реализует регистрацию клиента, изменение статуса и посещаемость. Методы обёрнуты декораторами, record_visit() использует контекстный менеджер, inspect_client() демонстрирует интроспекцию. Программа разделена на

логические компоненты, содержит проверку типичных ошибок и сопровождается автоматическими тестами.

Выполнил: Куршаков Егор Игоревич